COSC 470 Progress Report

Max Moir
University of Canterbury
Christchurch, New Zealand
maxwmoir@gmail.com

June 4, 2026

Abstract

The progress report (PDF with 3-5 pages excluding references; single-column text) should (obviously) summarise progress to date, and it should also outline remaining steps to project completion. Your examination committee comprises your supervisor and an examiner, who will be assigned to your project. They will assess your reports and presentations. Make sure you include enough introduction, background and motivation in all your reports and presentations so the examiner can understand them. Expect that your examiner is unfamiliar with your project and your research area.

1 Introduction

The mutual exclusion (mutex) problem, as originally introduced and solved by Dijkstra in [1], is a fundamental result in concurrent computing. Lamport's Bakery Algorithm (LBA) is a solution to the mutex problem published in [7]. Lamport elaborates on this in [8], analysing several solution variants.

For $N \leq 5$ processes, LBA can be verified using a model checker such as TLA+, but exhaustive state expansion is not tractable for higher values of N. The goal of this project is to develop a formal model of LBA generalised to any finite number of concurrent processes. The model will be implemented in Lean4 to ensure that it maintains several required properties.

1.1 Motivation

Distributed systems are difficult to implement correctly. Their distributed and concurrent nature can often lead to subtle bugs that are difficult to reproduce. Additionally, errors can propagate throughout a system, and partial failures can lead to unexpected negative outcomes. For this reason, it is often unrealistic to achieve total coverage with traditional testing. Formal verification can provide a strong guarantee of correctness in these situations. Interactive theorem proving as a verification method is especially strong for distributed systems, as it can verify correctness without explicitly exploring states. Theorem proving also allows for flexibility in the parameterisation of systems, such as verifying an algorithm for any arbitrary number of nodes, N. For this reason, this project will explore the verification of distributed algorithms with interactive theorem provers.

2 Background

The sections describes the problem statement as given in the research proposal, as well as an overview of the findings of my literature review. Additionally, it outlines the tools and methodologies chosen at the start of the project, and gives justification for using them.

2.1 Problem Statement

The mutex problem is summarized by Dijkstra in [1] as follows.

> To begin, consider N computers, each engaged in a process which, for our aims, can be regarded as cyclic. In each of the cycles a so-called "critical section" occurs and the computers have to be programmed in such a way that at any moment only one of these N cyclic processes is in its critical section.

A more detailed formalization of the problem was introduced by Lamport in [8], from which this proposal will take its terminology. Assuming that the ith process oscillates between critical and non-critical sections, we can represent its sequence of elementary executions as:

$$NCS_i^{[1]} \to CS_i^{[1]} \to NCS_i^{[2]} \to CS_i^{[2]} \to \dots$$

As explained in [8], there are multiple properties that must be shown to always hold to prove the correctness of the algorithm. This project will focus on the following two properties, in which $CS_i^{[k]}$ denotes the k-th execution of process i's critical section.

Definition 2.1 (Mutual Exclusion Property)**.** For any pair of distinct processes i and j, no pair of operation executions $CS_i^{[k]}$ and $CS_j^{[k']}$ are concurrent.

Definition 2.2 (Deadlock Freedom Property)**.** If there exists a looping process, then there exist an infinite number of critical section operation executions.

For a proof to be valid, these two properties must be shown to hold for every process in all state combinations. There are several variants of LBA that satisfy stronger requirements, but these are outside the scope of this project.

2.2 Related work

To gain an understanding of the current landscape of related work, I conducted a systematic literature review at the start of the project. Two major academic databases were selected: the ACM Digital Library and IEEE Xplore. These databases were chosen for their broad coverage of research in formal verification and distributed systems. The reference lists of the selected papers were scanned, and fundamental or commonly cited papers were added. Google Scholar was used as a supplementary tool for citation chaining. Any additional papers identified this way were included only if they met the above criteria or were primarily concerned with a related topic to be explained.

Many papers included were foundational problem or algorithm specifications [2][7][6][10]. Additionally many papers concerned frameworks for the verification of distributed algorithms in different languages. These include Verdi [15] and Disel [13] for Coq, Iron Fleet [3] for Dafny, and Veil [12] for Lean4. Some papers specifically focused on different variants of the bakery algorithm [4][5].

2.3 Tools

I selected to use Lean4 for this project because of its dual applications as a functional programming language and interactive theorem prover with extensive mathematical tooling. This lends it nicely to the verification of distributed protocols. It can be used equally to ensure invariants within programs, and to prove mathematical theorems. This project will use Lean4 to implement a distributed model of the transition system, show invariants hold within the state space and ensure that LBA guarentees mutual exculsion and deadlock freedom.

In 2025, the Veil [12] framework was introduced as a basis to verify distributed protocols specifically. Due to its novelty, there have been relatively few published attempts to verify algorithms with Lean4 outside of the Veil specification itself. Veil has both automated and interactive theorem proving capabilities built in. This means a user can interactively prove properties using native Lean tactics or use Veil's built-in SMT solvers. I chose to use Veil because it is a modern tool designed specifically to model the types of systems that I am interested in.

3 Progress to date

This section provides a summary of the current progress of the project. It explains the steps taken, and how these relate to the initially proposed methodology. The challenges faced so far in the project are also described in this section.

3.1 Work Completed

The first step taken towards completing this project was a comprehensive literature review, as described in the previous section.

After this, I used Veil to model a distributed transition system, formalizing the possible states and transitions of each node or process. Each state was given requirements for entry, as specified by the bakery algorithm description implemented in accordence with the Veil documentation. Listing

1 shows the Lean4 code to setup the initial state of each process, and define the possible transition actions between states.

The next step in verifying properties about this system was creating invariants which could be shown to hold under every transition. This process was aided will Veil's built in SMT (satisfiability modulo theories) solvers, which could quickly solve constraint problems to checker if a given variant could be shown to hold. SMT solvers are tools which can mechanically determine whether a proposition is satisfiable. These invariants needed to be built on top of each other, with the proof of one acting as a dependency in the verification of another. The goal of building these invariants was to show that the mutual exclusion safety property held between all transisitios. The Lean4 code to define these invariants can be seen in listing 2. At this point, the SMT solvers showed that mutual exclusion held under all transitions based on the previous definitions.

The next step was writing a human-readable formal proof for this. Veil can automatically generate the proof conditions and requirements from the transition system, so I used this to create a blank theorem with a pre-computed goal. This goal ensuring that mutual exclusion was preserved under transition into a process's critical state was previously automatically verified by the SMT solvers, but could also be manually verified. From here, I could use the assertions about the system, as well as the other invariants to show that mutual exclusion is guarenteed when transitioning into the critical state. This proof is shown in listing 3.

The work I've been exploring most recently is researching into verifying liveness. One framework which seems to show some promise is [14]. This approach looks promising for modelling the execution traces needed to validate liveness properties, similar to the approaches detailed in papers by Owicki et al. [11] and Yu et al. [16]. At this point, not enough progress has been made in this direction to include meaningful results here.

3.2 Challenges

One challenge I faced in the project was finding human understandable invariants that were useful for reasoning about the system. Often, specified invariants would either not be useful, or not be understandable for the reader. When I was first trying to find invariants, I wasn't using SMT solvers to verify that they held between states. Instead I used invariants similar to others I found in related literature. After discovering the "*#check_invariants*" feature of Veil, which uses SMT solvers to mechanically validate invariants, it was much easier to iterate. After discovering the SMT solvers, it was much easier to validate invariants, so I spent more time iterating on them to make them more intuative.

Another challenge I found was attempting to make the proof of mutual exclusion understandable. While using Lean4 you can keep track of the goal state that needs to be proved, and reduce it until each subcase can be proved. While I was reducing the proof, it was not always clear why a specific tactic reduced the goal state in the manner which it did. Due to this, the final proof that mutex holds does not resemble Lamport's proof in [8]. The formulation of this proof took longer than expected, and refactoring will be an ongoing process, because I'm not yet satisfied with the readability of the proof.

4 Remaining Work

The next steps and additional work required for project completion are detailed in this section. It provides a summary of outstanding tasks, a timeline for when each task should be completed, and a clear definition of project success.

4.1 Outstanding Tasks

The next main outstanding task in the project is proving liveness properties about the system. These are much more difficult to verify than the previously mentioned safety properties, and will require reasoning with temporal logic in order to verify. This connects back to the original aim of the project to verify deadlock freedom, which is a liveness property in Lamport's original specification of the Bakery algorithm. To ease this proof, it may be useful to specify additional invariants about the system. This is a dependency to liveness verification.

After deadlock freedom is verified for the bakery algorithm model, if there is sufficient time left in the project, verification of some properties of a similar algorithm might be possible. One potential candidate for extension is the Paxos consensus algorithm [9].

4.2 Timeline and Milestones

My future timeline and milestones will be based around the schedule given by the COSC470 course. After this progress report, the next examinable assessment is the oral presentation. Before this, I would like to have a formal proof for deadlock freedom in the system. Additionally, at this point a decision should be made on whether there is sufficient time to include any other algorithms in the project.

The next step after that is the final report submission. At this point, the entire project should be completed, including the verification of any potential additional algorithms. The final milestone is the demonstration of results. This comes after the final report is due, and a presentation should be constructed and practiced prior to this.

4.3 Definition of success

In order for this project to be successfull, the mutual exclusion and deadlock freedom properties must be shown to hold in the model. This is the minimum viable outcome for a successfull project. If any additional algorithms are included in the project, the relevant properties of them should be formally verified as well.

5 Conclusion

To summarise the progress to date, I have set up a model transition system representing the execution of the bakery algorithm on a single process, found invariants for several properties that can be shown to hold across all transitions, and provided a formal proof the mutual exclusion holds as a process enters its critical section. This progress aligns with the original aims and timeline set out at the beginning of the project. I have successfully modelled the transition system, and shown mutual exclusion to hold based on the requirements for entry into each state. At this point in the project, my outlook for meeting the success requirements is positive. So far, the project has met its goals for verifying safety properties in the system, and has a clear path forward.

References

[1] E. W. Dijkstra. Solution of a problem in concurrent programming control. *Communications of the ACM*, 8(9):569, Sept. 1965.

[2] E. W. Dijkstra. Solution of a problem in concurrent programming control. *Communications of the ACM*, 8(9):569, Sept. 1965.

[3] C. Hawblitzel, J. Howell, M. Kapritsos, J. R. Lorch, B. Parno, M. L. Roberts, S. Setty, and B. Zill. IronFleet: Proving practical distributed systems correct. In *Proceedings of the 25th Symposium on Operating Systems Principles*, SOSP '15, pages 1–17, New York, NY, USA, Oct. 2015. Association for Computing Machinery.

[4] W. H. Hesselink. Mechanical verification of Lamport's Bakery algorithm. *Science of Computer Programming*, 78(9):1622–1638, Sept. 2013.

[5] W. H. Hesselink. Correctness and concurrent complexity of the Black-White Bakery Algorithm. *Formal Aspects of Computing*, 28(2):325–341, Apr. 2016.

[6] L. Lamport. Safety, Liveness, and Fairness.

[7] L. Lamport. A new solution of Dijkstra's concurrent programming problem. *Commun. ACM*, 17(8):453–455, Aug. 1974.

[8] L. Lamport. The mutual exclusion problem: partII—statement and solutions. *Journal of the ACM*, 33(2):327–348, Apr. 1986.

[9] L. Lamport. Paxos made simple. *ACM SIGACT News*, 32(4):18–25, November 2001.

[10] L. Lamport, R. Shostak, and M. Pease. The Byzantine generals problem. In *Concurrency: The Works of Leslie Lamport*, pages 203–226. Association for Computing Machinery, New York, NY, USA, Oct. 2019.

[11] S. Owicki and L. Lamport. Proving Liveness Properties of Concurrent Programs. *ACM Transactions on Programming Languages and Systems*, 4(3):455–495, July 1982.

[12] G. Pîrlea, V. Gladshtein, E. Kinsbruner, Q. Zhao, and I. Sergey. Veil: A Framework for Automated and Interactive Verification of Transition Systems. In *Computer Aided Verification: 37th International Conference, CAV 2025, Zagreb, Croatia, July 23-25, 2025, Proceedings, Part III*, pages 26–41, Berlin, Heidelberg, July 2025. Springer-Verlag.

[13] I. Sergey, J. R. Wilcox, and Z. Tatlock. Programming and proving with distributed protocols. *Proc. ACM Program. Lang.*, 2(POPL):28:1–28:30, Dec. 2017.

[14] E. Vin, K. A. Miller, and D. J. Fremont. LeanLTL: A unifying framework for linear temporal logics in Lean. https://arxiv.org/abs/2507.01780v1, July 2025.

[15] J. R. Wilcox, D. Woos, P. Panchekha, Z. Tatlock, X. Wang, M. D. Ernst, and T. Anderson. Verdi: A framework for implementing and formally verifying distributed systems. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '15, pages 357–368, New York, NY, USA, June 2015. Association for Computing Machinery.

[16] X.-f. Yu and W. Hui. Formal verification of mutex of concurrent system. In *International Conference on Automatic Control and Artificial Intelligence (ACAI 2012)*, pages 1077–1082, Mar. 2012.

A Code Listings

```
-- Initial state

after_init {
  number N := 0;
  choosing N := False;
  critical N := False;
}

-- Transition actions

action choose (i : node) = {
    require ¬ choosing i;
    require number i = 0;
    choosing i := True;

    -- Find ticket value greater than all others
    let t_max <- fresh;
    require forall j, j ≠ i -> number j < t_max;
    number i := t_max;
    choosing i := False;
}

action enter (i : node) = {
    -- Only allow enter if not choosing
    require ¬ critical i;
    require number i ≠ 0;
    require forall j, j ≠ i -> ¬ choosing j;

    -- Only allow enter if i holds the smallest non-zero ticket number
    require forall j, j ≠ i ->
        (number j = 0) v
        (number i < number j) v
        (number i = number j ∧ lt i j);
    critical i := True;
}

action exit (i : node) = {
    require critical i
    critical i := False
    number i := 0
}
```

Listing 1: Initial state and transition actions

```
-- Invariants

safety [mutex]
    ∀ I J, critical I → critical J → I = J


invariant [different_vals]
    number I ≠ 0 →
        number I = number J →
        I = J


invariant [critical_lowest]
    critical I →
        ∀ J, J ≠ I → number J = 0 ∨ number I < number J


invariant [critical_has_ticket]
    critical I →
        number I ≠ 0
```

Listing 2: System invariants

```
@[invProof]
theorem enter_mutex1 :
    ∀ (st : @State node),
      ∀ (i : node),
        (@System node node_dec node_ne tot).assumptions st →
          (@System node node_dec node_ne tot).inv st →
            (@Mutex.enter.ext node node_dec node_ne tot i) st fun _ (st' : @State
                node) =>
              @Mutex.mutex node node_dec node_ne tot st' :=
  by

    -- Move state, process trying to enter i, and invariant into the Lean context.
    intros st i assumptions inv

    -- Unfold goal and invariant definitions.
    simp [Mutex.enter.ext, invSimp] at *

    -- Split invariant into individual clauses.
    rcases inv with ⟨critical_has_ticket, critical_lowest, different_vals, mutex⟩

    -- Add implictations to lean context
    rintro h_not_critical h_num h_choose h_lowest N M critN critM

    -- Now we need to prove if two arbitrary processes N and M are both in their
    -- critical sections, then N = M
    -- The currently entering process is i

    -- Use our different_vals invariant to split the current goal
    apply different_vals

    -- Show that number N ≠ 0
    · by_cases h : (N = i)
      -- If N = i
      · simp [h]
        exact h_num
      -- If N ≠ i
      · simp [h] at critN
        exact critical_has_ticket N critN

    -- Show that number N = number M
    · by_cases hN : N = i <;> by_cases hM : M = i
      -- N = i and M = i
      · simp [hN, hM]

      -- N = i and M ≠ i
      · simp [hM] at critM
        have hMlow := critical_lowest M critM i (Ne.symm hM)
        rcases hMlow with h0 | hlt
        · simp [hN, h0]
          omega
        · have hilow := h_lowest M hM
```

```
          rcases hilow with h0 | hlt' | ⟨heq, _⟩
          · simp [hN] at critN
            exact absurd h0 (critical_has_ticket M critM)
          · omega
          · omega

      -- N ≠ i and M = i
      · simp [hN] at critN
        have hNlow := critical_lowest N critN i (Ne.symm hN)
        rcases hNlow with h0 | hlt
        · simp [hM, h0]
          omega
        · have hilow := h_lowest N hN
          rcases hilow with h0 | hlt' | ⟨heq, _⟩
          · simp [hM] at critM
            exact absurd h0 (critical_has_ticket N critN)
          · omega
          · omega

      -- N ≠ i and M ≠ i
      · simp [hN] at critN
        simp [hM] at critM
        have same : N = M := by exact mutex N M critN critM
        simp [same]
```

Listing 3: Proof that the `enter` action preserves the mutex safety property.